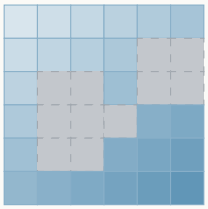


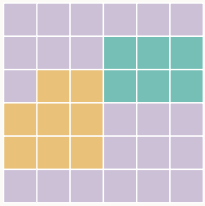
(T, H, W, C)

UNE VIDÉO



$x \longrightarrow E_\theta(x)$

ON EN CACHE



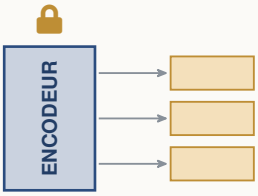
$\|\hat{z} - z\|_1$

ON DEVINE L'IDÉE

V-JEPA 2.1 pas à pas

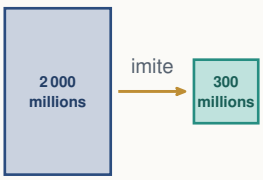
Comprendre, coder et déployer un modèle vidéo auto-supervisé, en partant vraiment de zéro

Aucun prérequis : on part vraiment de zéro.



une sonde par tâche

ENCODEUR GELÉ



presque la même qualité

DISTILLATION

Doctor Mokira

2026

Édition 2026. Document composé avec LaTeX (pdfTeX), polices Palatino (texte et mathématiques), TeX Gyre Heros (titres) et DejaVu Sans Mono (code). Figures et schémas réalisés avec TikZ et pgfplots, coloration syntaxique assurée par minted/Pygments, encadrés par tcolorbox, bibliographie par biblatex.

Ce livre explique de bout en bout l'article *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning*, publié par l'équipe FAIR de Meta et l'Université de Saragosse. Il est écrit pour être lu sans aucun prérequis : on part de zéro, on explique avec des mots avant d'écrire des symboles, et on n'avance jamais sans avoir d'abord dit pourquoi. Il ne s'arrête pas à la compréhension : il conduit le lecteur jusqu'au code qui tourne, puis jusqu'au modèle mis en service.

Avant-propos

« Un article de recherche ne devrait effrayer personne. »

Ce livre raconte un seul article scientifique, du premier mot jusqu'à la dernière ligne de code : *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning* [2], écrit par l'équipe FAIR de Meta et l'Université de Saragosse. Cet article présente un programme qui apprend à comprendre des vidéos tout seul, sans qu'un humain lui explique jamais ce qu'il regarde. Il fait suite à un premier modèle, V-JEPA 2 [1], dont nous parlerons beaucoup : c'est en corrigeant ses défauts que V-JEPA 2.1 est né.

Un article de recherche est écrit par des spécialistes, pour des spécialistes. Il va vite. Il emploie des mots que personne ne vous a jamais définis. Il suppose que vous connaissez déjà dix autres articles. Résultat : beaucoup de gens curieux, capables, motivés, s'arrêtent à la première page.

Ici, on ne suppose rien du tout. Chaque idée est d'abord racontée avec des mots de tous les jours, puis dessinée, puis seulement écrite en langage mathématique, puis enfin mise en pratique avec du code que vous pouvez faire tourner vous-même.

> **En clair**
>
> Notre promesse : si vous savez compter et si vous savez lire, vous pouvez comprendre comment fonctionne V-JEPA 2.1. Le reste, nous le construisons ensemble, dans l'ordre, sans jamais sauter une marche.

Pour qui est ce livre

Pour toute personne qui veut trois choses à la fois :

1. **comprendre** ce que fait V-JEPA 2.1, et pourquoi il fait mieux que le modèle qui existait avant lui ;

2. **écrire elle-même** le modèle et le programme qui l'entraîne ;
3. **mettre le modèle au travail** pour de vrai, tous les jours, au service d'une application.

Aucun diplôme n'est demandé. Vous n'avez pas besoin de savoir ce qu'est un réseau de neurones, ni un transformeur, ni même une image numérique : tout cela est expliqué à partir de rien, dans les quatre premiers chapitres.

Comment ce livre est fabriqué

Huit règles ont guidé chaque page, et elles ne sont pas des intentions : ce sont des engagements que vous pouvez vérifier.

1. Aucun mot technique n'apparaît sans avoir été défini juste avant.
2. On commence toujours par les prérequis, jamais par la nouveauté.
3. Un dessin pour chaque idée qui bouge, qui circule ou qui se compare.
4. Un exemple après chaque notion, avec des chiffres quand c'est possible.
5. Des mots simples, ceux de tous les jours.
6. Le texte vous parle et vous accompagne.
7. Une phrase de transition à la fin de chaque section, qui annonce la suite.
8. Des phrases entières, comme dans un vrai livre.

Le texte utilise quelques encadrés qui reviennent souvent, faciles à reconnaître à leur pictogramme :

- **En clair** — l'idée expliquée avec des mots de tous les jours.
- **L'intuition** — l'image mentale à garder en tête.
- **À quoi ça sert en IA ?** — où cette notion sert vraiment dans V-JEPA 2.1.
- **Sur des données réelles** — la même idée, appliquée aux chiffres réels de l'article.
- **Ce qu'il faut savoir avant** — les seuls prérequis de la section.
- **Piège à éviter** — l'erreur classique que tout le monde fait.
- **À retenir** — l'essentiel, si vous ne deviez garder qu'une chose.

- 🚀 **Pour aller plus loin** — un pont vers un sujet plus avancé, que vous pouvez sauter en première lecture.

Chaque chapitre se termine par des exercices, tous corrigés. Ils ne servent pas à vous noter : ils servent à vous montrer, à vous-même, que vous avez compris.

Un mot d'honnêteté

L'article explique comment *entraîner* le modèle et comment l'*évaluer*. Il ne parle presque pas de sa *mise en service* : pas de serveur, pas de temps de réponse, pas de surveillance. La dernière partie de ce livre sera donc de deux couleurs, et nous vous le dirons franchement à chaque fois :

- **Ce que l'article dit** — la taille des modèles, la résolution des images, la vitesse de calcul. Ce sont des résultats mesurés.
- **Ce que nous ajoutons** — le serveur, la mesure du temps de réponse, la surveillance. Ce sont des conseils de métier, utiles et éprouvés, mais qui ne viennent pas de l'article.

Nous dirons aussi, aussi clairement que ses réussites, ce que ce modèle ne sait pas faire, et les tâches sur lesquelles d'autres modèles restent meilleurs que lui. Un livre qui ne raconte que les victoires ne vous aide pas à choisir.

Bonne lecture.

Table des matières

Avant-propos iii
Table des matières vii

I Les fondations : partir vraiment de zéro 1
1 Une image, une vidéo, vues par un ordinateur 3
1.1 Un ordinateur ne voit pas des objets, il voit des nombres . . . 4
1.2 La résolution : la finesse de la mosaïque 6
1.3 La couleur, c'est trois grilles au lieu d'une 7
1.4 Ranger les nombres : le tenseur et sa forme 9
1.5 Une vidéo, c'est une pile d'images 11
1.6 Ce que tout cela pèse vraiment 14
1.7 Passons au code 16
Exercices 18
2 Ce que veut dire « apprendre » pour une machine 21
2.1 Une machine qui apprend, c'est une machine à boutons 22
2.2 Se tromper, et mesurer de combien 25
2.3 De quel côté tourner le bouton ? La pente 28
2.4 La descente de gradient, calculée à la main 30
2.5 La taille des pas : le taux d'apprentissage 34
2.6 Le rythme de l'entraînement : le lot et l'époque 36
2.7 Apprendre par cœur n'est pas apprendre 39
2.8 Passons au code 40
Exercices 44

II Comprendre JEPA et V-JEPA 2, l'ancêtre direct 49

III Le problème que V-JEPA 2.1 vient résoudre 51

IV V-JEPA 2.1, ingrédient par ingrédient 53

V Les résultats, lus avec honnêteté 55

VI Construire le modèle et le programme d'entraînement 57

VII De l'entraînement à la production 59

Bibliographie 61

Première partie

Les fondations : partir vraiment de zéro

CHAPITRE 1

Une image, une vidéo, vues par un ordinateur

Avant de parler d'intelligence artificielle, de vidéos cachées et de modèles à deux milliards de réglages, mettons-nous d'accord sur une chose toute simple : qu'est-ce qu'une image, pour une machine ? Ce chapitre pose le décor, tranquillement, sans rien supposer.

> **Ce qu'il faut savoir avant**
>
> Rien du tout. Vraiment rien. Il faut savoir compter, savoir qu'une multiplication sert à répéter une addition, et c'est tout. Chaque mot nouveau sera expliqué au moment où il arrive.

Vous regardez une photo de chat. Vous voyez un chat. C'est immédiat, c'est gratuit, vous n'avez rien eu à faire. Un ordinateur, lui, ne voit pas de chat. Il ne voit même pas d'image. Il voit une très longue liste de nombres, et rien d'autre.

Tout ce livre part de là. Si nous voulons qu'une machine finisse par comprendre une vidéo, il faut d'abord savoir sous quelle forme la vidéo arrive chez elle. C'est exactement ce que nous allons faire dans ce chapitre : ouvrir le capot et regarder les nombres.

1.1 Un ordinateur ne voit pas des objets, il voit des nombres

En clair

Une image, pour un ordinateur, c'est une grille de petits carrés. Chaque carré porte un nombre, et ce nombre dit une seule chose : à quel point ce petit carré est clair ou sombre.

Pensez à une mosaïque, comme celles qu'on voit sur les murs des vieilles maisons. De loin, vous voyez un dessin. De près, vous voyez des centaines de petits carreaux collés les uns aux autres, chacun d'une seule couleur. Une image numérique, c'est exactement cela : une mosaïque de petits carrés, tous de la même taille, rangés en lignes et en colonnes.

Définition 1.1 (Le pixel)

Un *pixel* est un de ces petits carrés. C'est le plus petit morceau d'une image : on ne peut pas le couper en deux. Le mot vient de l'anglais *picture element*, c'est-à-dire « élément d'image ». Chaque pixel porte un nombre qui décrit sa couleur.

Commençons par le cas le plus simple : une image en *niveaux de gris*, c'est-à-dire une image sans couleur, comme une vieille photo en noir et blanc. Dans ce cas, un seul nombre suffit par pixel. On se met d'accord une fois pour toutes sur ce que ce nombre veut dire :

- le nombre **0** veut dire « ce carré est complètement noir » ;
- le nombre **255** veut dire « ce carré est complètement blanc » ;
- tous les nombres entre les deux sont des gris, de plus en plus clairs à mesure qu'on monte.

La figure 1.1 montre les deux faces de la même image : à gauche ce que la machine a réellement en mémoire, à droite ce que nous voyons apparaître à l'écran.

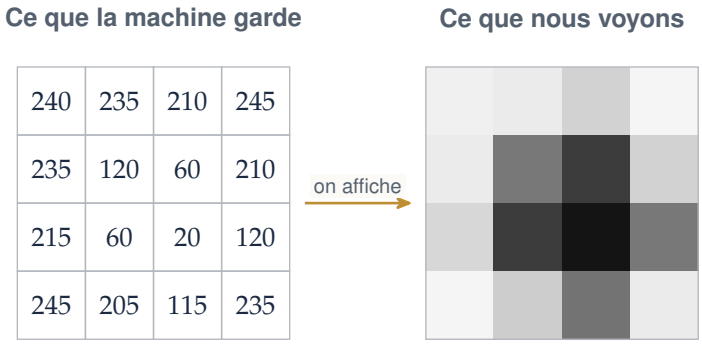


FIGURE 1.1. Les deux visages d'une même image. À gauche, les seize nombres que l'ordinateur garde en mémoire. À droite, le dessin obtenu en peignant chaque carré : petit nombre, carré sombre ; grand nombre, carré clair. Il n'y a rien d'autre dans une image que ces nombres.

Exemple 1.2 (Lire trois pixels)

Dans la grille de gauche de la figure 1.1, le pixel tout en haut à gauche vaut 240 : il est presque blanc. Le pixel du milieu vaut 20 : il est presque noir. Un pixel qui vaudrait 128, soit à peu près la moitié de 255, serait un gris moyen, ni clair ni foncé.

L'intuition

Voyez chaque pixel comme un petit interrupteur à gradateur, du genre de ceux qui règlent la lumière d'un salon. Tourné à fond dans un sens, la lampe est éteinte : c'est 0. Tourné à fond dans l'autre, la lampe éclaire au maximum : c'est 255. Une image, c'est un mur couvert de milliers de ces petites lampes, chacune réglée à sa valeur.

Piège à éviter

Pourquoi 255 et pas 100, ou 1000 ? C'est une simple convention, née de la façon dont les ordinateurs rangent les nombres : avec huit cases de mémoire, on peut écrire exactement 256 valeurs différentes, de 0 à 255. Retenez surtout le sens de lecture : **plus le nombre est grand, plus le pixel est clair**. Beaucoup de débutants inversent, dessinent des images

en négatif, et cherchent l'erreur pendant une heure.

Nous savons maintenant ce que contient une case de la mosaïque. La question suivante vient toute seule : combien y a-t-il de cases ?

1.2 La résolution : la finesse de la mosaïque

Définition 1.3 (La résolution)

La *résolution* d'une image, c'est le nombre de pixels qu'elle contient, donné sous la forme « largeur × hauteur ». Une image de résolution 256×256 possède 256 colonnes de pixels et 256 lignes de pixels.

Plus la mosaïque a de carreaux, plus le dessin est fin. Avec très peu de carreaux, un rond ressemble à un escalier ; avec beaucoup, il redevient un rond. La figure 1.2 montre le même disque dessiné deux fois, la première fois grossièrement, la seconde finement.

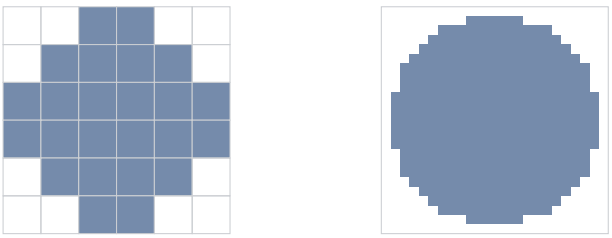


FIGURE 1.2. Le même disque, dessiné avec deux finesses différentes. À gauche, la mosaïque est grossière et les bords deviennent des marches d'escalier. À droite, les carreaux sont seize fois plus nombreux et l'œil retrouve un vrai rond. Rien n'a changé dans le dessin : seule la finesse de la grille a changé.

Le nombre total de pixels s'obtient par une simple multiplication : on multiplie la largeur par la hauteur, comme on calculerait le nombre de carreaux d'un sol.

Exemple 1.4 (Compter les pixels d'une vraie image)

Une image de résolution 256×256, qui est la taille utilisée pendant la plus grande partie de l'entraînement de V-JEPA 2.1, contient

$$256 \times 256 = 65\,536 \text{ pixels.}$$

Soit soixante-cinq mille petits carrés. Et ce n'est encore qu'une image en gris : nous n'avons même pas parlé de couleur.

À retenir

Deux images peuvent montrer exactement la même chose et ne pas avoir la même résolution. Quand la résolution monte, le dessin gagne en finesse, mais le nombre de nombres à traiter grimpe très vite : il grimpe comme le *carré* de la taille. Doubler la largeur et la hauteur, c'est multiplier le travail par quatre. Cette remarque, qui paraît anodine ici, expliquera plus tard pourquoi la dernière phase d'entraînement de V-JEPA 2.1 est si courte.

Nos images sont encore grises. Ajoutons la couleur : vous allez voir que cela ne change presque rien à l'histoire.

1.3 La couleur, c'est trois grilles au lieu d'une

Un écran ne sait fabriquer que trois couleurs de lumière : du rouge, du vert et du bleu. Toutes les autres couleurs sont obtenues en mélangeant ces trois-là, en dosant chacune plus ou moins fort. Du rouge à fond plus du vert à fond donne du jaune. Les trois à fond donnent du blanc. Les trois éteintes donnent du noir.

Un pixel en couleur a donc besoin non pas d'un nombre, mais de trois : un pour le rouge, un pour le vert, un pour le bleu. Chacun se lit comme avant, de 0 (éteint) à 255 (à fond).

Définition 1.5 (Le canal)

Un *canal* est une grille complète de nombres qui ne s'occupe que d'une seule couleur de base. Une image en couleur possède trois canaux superposés : le canal rouge, le canal vert et le canal bleu. On les appelle souvent « les canaux R, V, B », ou « RGB » à l'anglaise.

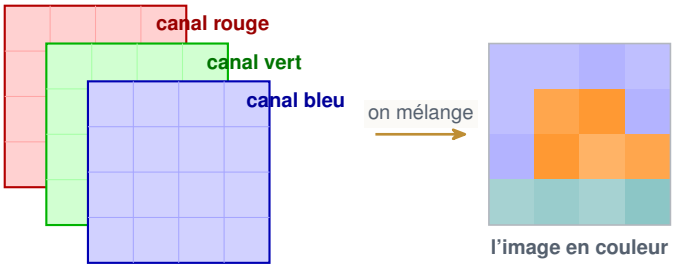


FIGURE 1.3. Une image en couleur, c'est trois mosaïques posées les unes sur les autres. La première dit combien il y a de rouge dans chaque carré, la deuxième combien de vert, la troisième combien de bleu. En regardant les trois ensemble, l'œil reconstitue la couleur.

Exemple 1.6 (Quelques couleurs, en trois nombres)

Voici comment se lisent quelques couleurs courantes, dans l'ordre rouge, vert, bleu :

Couleur	Rouge	Vert	Bleu
Noir	0	0	0
Blanc	255	255	255
Rouge vif	255	0	0
Jaune	255	255	0
Gris moyen	128	128	128

Vous remarquez que le gris s'obtient en donnant la même valeur aux trois canaux. C'est logique : aucune couleur ne tire la couverture à elle.

Comptons maintenant, pour de bon, le nombre de cases d'une image en couleur.

Exemple 1.7 (Le poids d'une image de 256×256)

Notre image possède 65 536 pixels, et chaque pixel réclame trois nombres. Au total :

$$256 \times 256 \times 3 = 196\,608 \text{ nombres.}$$

Presque deux cent mille nombres pour une seule petite image carrée, plus petite que la moitié d'un écran de téléphone. Gardez ce chiffre en tête : il va grossir très vite.

Nous avons beaucoup de nombres sur les bras. Il est temps de les ranger proprement, et de leur donner un nom.

1.4 Ranger les nombres : le tenseur et sa forme

Pour parler de tous ces nombres sans se perdre, les informaticiens utilisent un mot unique, qui a l'air savant mais qui ne cache rien de compliqué : le *tenseur*. Approchons-le par étapes, en montant une marche à la fois.

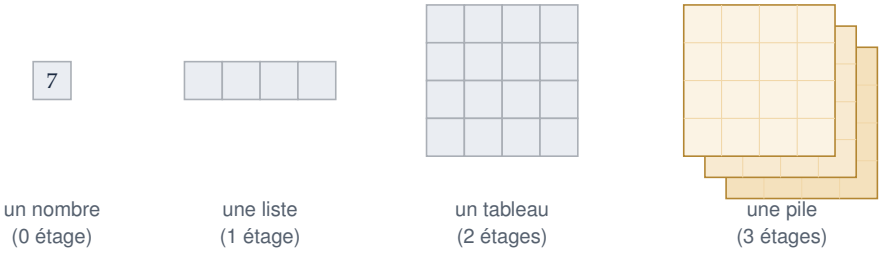


FIGURE 1.4. La même idée, à quatre étages. Un nombre tout seul ; une liste de nombres ; un tableau de nombres ; puis une pile de tableaux. Le mot « tenseur » désigne tout cela d'un coup, quel que soit le nombre d'étages.

Définition 1.8 (Le tenseur)

Un *tenseur* est un tableau de nombres qui peut avoir autant de directions que l'on veut. Un nombre seul est un tenseur sans direction ; une liste est un tenseur à une direction ; un tableau à lignes et colonnes est un tenseur à deux directions ; une pile de tableaux est un tenseur à trois directions, et ainsi de suite. Chaque direction s'appelle un *axe*.

Pour décrire un tenseur, il suffit de dire combien de cases il compte le long de chaque axe. Cette liste de tailles porte un nom, et c'est le mot le plus utile de tout le chapitre.

Notation 1.9 (La forme d'un tenseur)

La *forme* d'un tenseur est la liste du nombre de cases sur chaque axe. On l'écrit entre parenthèses. Pour une image en couleur, on écrit

$$(\text{hauteur},\ \text{largeur},\ \text{canaux}),$$

et notre image de tout à l'heure a donc la forme $(256, 256, 3)$.

Exemple 1.10 (Lire une forme)

Un tenseur de forme $(64, 384, 384, 3)$ se lit de gauche à droite comme une phrase : « soixante-quatre choses, chacune haute de 384 cases, large de 384 cases, et épaisse de 3 canaux ». Nous verrons dans un instant que ces soixante-quatre choses sont les images d'une vidéo.

Piège à éviter

L'ordre des axes n'est pas le même partout, et c'est la source d'erreur numéro un des débutants. Deux conventions se partagent le monde :

- $(\text{hauteur},\ \text{largeur},\ \text{canaux})$ — l'ordre des outils d'affichage et de photo ;
- $(\text{canaux},\ \text{hauteur},\ \text{largeur})$ — l'ordre des bibliothèques d'apprentissage comme PyTorch.

Les nombres sont exactement les mêmes, seule leur façon d'être rangés change. Mais si vous vous trompez, votre programme croira qu'une image de 256×256 en couleur est une image de 3 pixels de haut sur 256 de large avec 256 canaux, et vous obtiendrez une bouillie. Prenez l'habitude, dès aujourd'hui, d'écrire la forme attendue en commentaire au-dessus de chaque ligne de code qui manipule une image.

Nous savons ranger une image. Passons à ce qui nous intéresse vraiment dans ce livre : la vidéo.

1.5 Une vidéo, c'est une pile d'images

Une vidéo n'est rien d'autre qu'une suite d'images fixes, montrées les unes après les autres, très vite. Si elles défilent assez vite, notre œil ne voit plus des images séparées : il voit du mouvement. C'est le principe du petit carnet à feuilleter, celui où l'on dessine un bonhomme dans le coin de chaque page et où l'on fait défiler les pages avec le pouce.

Définition 1.11 (L'image d'une vidéo)

Chacune des images fixes qui composent une vidéo s'appelle une *image*, ou une *trame* ; en anglais on dit *frame*, et ce mot revient tellement souvent dans les articles et dans le code qu'il faut le connaître. Une vidéo est donc une pile d'images empilées non pas dans l'espace, mais dans le temps.

Définition 1.12 (Les images par seconde)

Le nombre d'*images par seconde* dit combien d'images sont montrées pendant une seconde. On l'abrège souvent en *fps*, de l'anglais *frames per second*. Un film de cinéma tourne à 24 images par seconde ; les vidéos de téléphone, souvent à 30 ou 60.

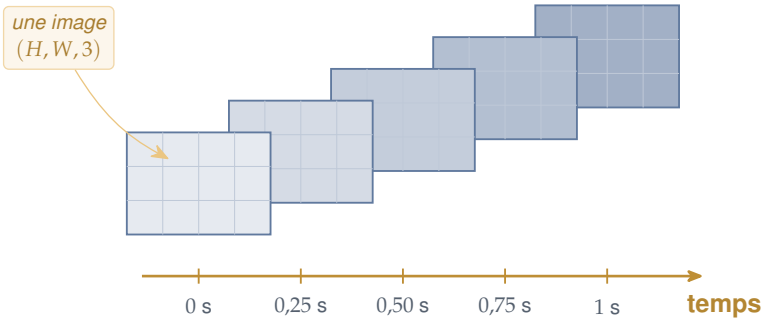


FIGURE 1.5. Une vidéo vue comme une pile de feuilles. Chaque feuille est une image complète, avec ses trois canaux de couleur. La flèche du bas est le temps : elle avance d'un cran à chaque nouvelle feuille. Ici, quatre images par seconde, donc un quart de seconde entre deux feuilles.

Puisque la vidéo est une pile d'images, sa forme s'obtient en ajoutant un axe

devant celle d'une image : l'axe du temps.

Notation 1.13 (La forme d'une vidéo)

Un extrait de vidéo s'écrit sous la forme

$$(\text{temps},\ \text{hauteur},\ \text{largeur},\ \text{canaux}),$$

où le premier nombre est le nombre d'images de l'extrait. On appelle souvent un tel extrait un *clip* : c'est un petit morceau découpé dans une vidéo plus longue, exactement comme on découpe un morceau dans une bobine de film.

La durée d'un clip se calcule sans effort. Si vous avez un certain nombre d'images, et que vous savez combien d'images passent en une seconde, alors la division vous donne les secondes :

$$\text{durée en secondes} = \frac{\text{nombre d'images}}{\text{images par seconde}}.$$

Exemple 1.14 (La durée d'un clip d'entraînement)

Pendant la plus grande partie de son entraînement, V-JEPA 2.1 regarde des clips de 16 images prises à raison de 4 images par seconde. Ces clips durent donc

$$\frac{16}{4} = 4 \text{ secondes}.$$

Quatre secondes, c'est court, mais c'est bien assez pour voir un geste commencer et se terminer : une main qui saisit une tasse, une porte qui s'ouvre. Et c'est justement ce que le modèle doit apprendre à comprendre.

Sur des données réelles

L'article de V-JEPA 2.1 utilise deux réglages, l'un après l'autre. Pendant la longue phase d'entraînement principale, les clips font 16 images en 256×256. Pendant la courte phase finale, appelée phase de refroidissement, on monte à 64 images en 384×384. Toujours à quatre images

par seconde, ces clips-là durent

$$\frac{64}{4} = 16 \text{ secondes.}$$

Le modèle regarde donc, à la fin, des extraits quatre fois plus longs et beaucoup plus fins. Nous verrons au chapitre consacré au refroidissement pourquoi cet ordre — d'abord grossier et court, ensuite fin et long — n'est pas un caprice.

Comptons enfin les nombres contenus dans un clip, en multipliant tout simplement les quatre tailles entre elles.

Exemple 1.15 (Le nombre de cases d'un clip)

Pour un clip de 16 images en 256×256 couleur :

$$16 \times 256 \times 256 \times 3 = 3\,145\,728 \text{ nombres.}$$

Plus de trois millions. Et pour un clip de la phase de refroidissement, 64 images en 384×384 :

$$64 \times 384 \times 384 \times 3 = 28\,311\,552 \text{ nombres,}$$

soit environ vingt-huit millions, c'est-à-dire neuf fois plus que le précédent.

À retenir

Une image est un tenseur à trois axes, une vidéo un tenseur à quatre axes. Rien de plus mystérieux : on a seulement ajouté le temps devant. Toutes les formes que vous croiserez dans ce livre se lisent avec cette seule règle.

Trois millions de nombres, cela reste abstrait. Regardons ce que cela représente vraiment dans la mémoire d'une machine.

1.6 Ce que tout cela pèse vraiment

Un ordinateur range ses nombres dans des cases de mémoire. L'unité de mesure de cette mémoire est l'*octet*.

Définition 1.16 (L'octet)

Un *octet* est un petit paquet de mémoire, celui qui suffit tout juste à écrire un nombre entier compris entre 0 et 255. C'est aussi l'unité dans laquelle on mesure la taille des fichiers. Mille octets font environ un kilo-octet, un million d'octets font environ un méga-octet (noté Mo), un milliard d'octets font environ un giga-octet (noté Go).

Nos pixels vont de 0 à 255 : ils tiennent donc chacun dans un seul octet. Mais dès qu'un modèle se met à calculer, il a besoin de nombres à virgule — parce qu'une moyenne, une multiplication ou une petite correction tombent rarement sur un compte rond. Ces nombres à virgule s'appellent des *nombres flottants*, et le format le plus courant en occupe quatre octets chacun, soit quatre fois plus.

Exemple 1.17 (Du fichier à la mémoire de calcul)

Notre clip de 3 145 728 nombres pèse :

- 3 145 728 octets, soit environ **3 Mo**, tant que les pixels restent des entiers de 0 à 255 ;
- $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ **12,6 Mo**, une fois convertis en nombres à virgule pour le calcul.

Or un modèle ne regarde jamais un seul clip à la fois : il en regarde tout un paquet d'un coup, pour aller plus vite. Ce paquet s'appelle un *lot*, et sa taille est un réglage du programme d'entraînement.

Exemple 1.18 (Le poids d'un lot entier)

V-JEPA 2.1 traite 128 clips à la fois. Pour la phase principale, cela donne

$$128 \times 12\,582\,912 \approx 1\,610\,000\,000 \text{ octets},$$

soit environ **1,6 Go** — rien que pour les vidéos d'entrée, avant même le

moindre calcul. Pour la phase de refroidissement, où chaque clip pèse neuf fois plus, on arrive à environ **14,5 Go**. Voilà, en une ligne de calcul, pourquoi cette phase-là ne dure que très peu de temps : elle coûte cher.

À quoi ça sert en IA ?

Ce chapitre a l'air de ne parler que de pixels, mais il pose déjà les trois problèmes que tout le reste du livre s'emploie à résoudre.

- **Il y a beaucoup trop de nombres.** Aucun modèle ne peut traiter trois millions de valeurs une par une. Il faudra donc d'abord les regrouper en petits carrés de 16×16 pixels : c'est ce que nous ferons au chapitre 4, consacré au transformeur de vision.
- **La finesse coûte cher.** Chaque fois que la résolution monte, la facture explose. C'est pour cela que l'entraînement commence en basse résolution et ne monte qu'à la toute fin.
- **Lire une vidéo prend du temps.** Une vidéo est stockée compressée ; il faut la décompresser image par image avant de pouvoir en faire quoi que ce soit. En production, ce travail-là devient souvent plus lent que le modèle lui-même. Nous y reviendrons longuement dans la partie consacrée à la mise en service.

Pour aller plus loin

Dans le même temps où il regarde 128 clips, V-JEPA 2.1 regarde aussi 2 304 images fixes. En reprenant nos calculs, ces images pèsent $2\,304 \times 196\,608 \times 4 \approx 1{,}8$ Go, c'est-à-dire à peu près autant que les 1,6 Go de vidéo. Ces deux nombres, 128 et 2 304, n'ont donc pas été tirés au hasard : ils ont été choisis ensemble pour que les deux moitiés du travail coûtent à peu près pareil. Nous y reviendrons au chapitre sur les données.

Assez de calculs sur le papier. Vérifions tout cela sur une vraie machine.

1.7 Passons au code

Nous allons manipuler nos images et nos clips avec Python, le langage utilisé par presque tous les travaux d'intelligence artificielle, et avec `numpy`, la bibliothèque qui sait manipuler les tenseurs. Vous n'avez pas besoin de comprendre chaque ligne dès maintenant : lisez d'abord les commentaires, et comparez les nombres affichés avec ceux que nous venons de calculer à la main.

PYTHON • Fabriquer une image et lire sa forme

```python
import numpy as np

# Une image en couleur de 256 x 256 pixels, entierement noire.
# La forme se lit : (hauteur, largeur, canaux).
image = np.zeros((256, 256, 3), dtype=np.uint8)

print(image.shape)   # -> (256, 256, 3)
print(image.size)    # -> 196608   le nombre total de cases
print(image.dtype)   # -> uint8    un entier de 0 a 255
print(image.nbytes)  # -> 196608   un octet par case

# Peignons le pixel de la ligne 10, colonne 20, en rouge vif.
image[10, 20] = [255, 0, 0]
print(image[10, 20])  # -> [255   0   0]
```

Le mot `uint8` se lit « entier non signé sur huit bits » : c'est exactement le type de nombre qui va de 0 à 255, celui dont nous avons parlé. Passons maintenant à la vidéo.

PYTHON • Un clip, sa forme, sa durée et son poids

```python
import numpy as np

# Un clip de 16 images de 256 x 256 en couleur.
# La forme se lit : (temps, hauteur, largeur, canaux).
clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)

print(clip.shape)   # -> (16, 256, 256, 3)
print(clip.size)    # -> 3145728   les trois millions annonces

images_par_seconde = 4.0
```

```
duree = clip.shape[0] / images_par_seconde
print(duree, "secondes")            # -> 4.0 secondes

# Le meme clip converti en nombres a virgule, ramenes entre 0 et 1.
clip_flottant = clip.astype(np.float32) / 255.0
print(clip_flottant.nbytes / 1e6, "Mo")   # -> 12.582912 Mo
```

Enfin, voici le changement d'ordre des axes dont nous parlions plus haut. C'est une opération que vous ferez des centaines de fois, alors autant la voir tout de suite.

PYTHON • Passer de l'ordre « affichage » à l'ordre « PyTorch »

```python
import numpy as np
import torch

clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)   # (T, H, W, C)

x = torch.from_numpy(clip).float() / 255.0
x = x.permute(3, 0, 1, 2)                            # (C, T, H, W)

print(x.shape)   # -> torch.Size([3, 16, 256, 256])

# Aucun nombre n'a ete perdu : seul l'ordre des axes a change.
print(x.numel())  # -> 3145728
```

Piège à éviter

Remarquez la dernière ligne : le nombre de cases n'a pas bougé. Changer l'ordre des axes ne crée et ne détruit aucune donnée, cela réorganise seulement la façon de les parcourir. Si jamais ce nombre change après une telle opération, c'est que vous avez commis une erreur ailleurs — et cette vérification en une ligne vous fera gagner des heures.

À retenir

Une image est une grille de nombres ; la couleur ajoute une troisième dimension ; la vidéo en ajoute une quatrième, le temps. Tout le reste de ce livre travaille sur ces tenseurs-là, et sur rien d'autre. Quand vous serez perdu dans un chapitre plus difficile, revenez à cette question toute simple : « quelle est la forme de ce que je manipule ? » Elle remet presque toujours les idées en place.

Nous savons désormais sous quelle forme une image et une vidéo arrivent dans une machine. Mais nous n'avons encore rien appris à personne : nos tenseurs restent inertes. La question suivante est donc la bonne : qu'est-ce que cela veut dire, au juste, qu'une machine « apprend » ? C'est tout le sujet du chapitre suivant.

Exercices

Exercice 1 — Compter les cases

Combien de nombres contient une image en couleur de 512×512 pixels ? C'est la taille des images utilisées pendant la phase de refroidissement de V-JEPA 2.1.

Solution

On multiplie la hauteur, la largeur et le nombre de canaux : $512 \times 512 \times 3 = 262\,144 \times 3 = 786\,432$ nombres. C'est quatre fois plus qu'une image de 256×256, ce qui est normal puisque nous avons doublé les deux côtés.

Exercice 2 — La durée d'un clip

Un clip contient 32 images, prises à raison de 8 images par seconde. Combien de temps dure-t-il ? Et combien de temps durerait-il si les mêmes 32 images étaient prises à 4 images par seconde ?

Solution

Dans le premier cas, $32/8 = 4$ secondes. Dans le second, $32/4 = 8$ secondes. Le nombre d'images est le même, mais elles sont prélevées plus loin les unes des autres : le clip couvre donc une tranche de temps deux fois plus longue. C'est un réglage important, car il décide de ce que le modèle a le droit de voir.

Exercice 3 — Le poids d'un lot

Un lot contient 64 clips de 16 images en 256×256 couleur, convertis en nombres à virgule de quatre octets. Combien de méga-octets occupe ce lot ?

Solution

Un clip contient $16 \times 256 \times 256 \times 3 = 3\,145\,728$ nombres, donc $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ 12,6 Mo. Pour 64 clips : $64 \times 12{,}6 \approx 806$ Mo, c'est-à-dire un peu plus de huit cents méga-octets. Ce lot fait exactement la moitié de celui de l'article, qui en contient 128, et l'on retrouve bien les 1,6 Go calculés plus haut en doublant ce résultat.

Exercice 4 — Lire une forme

Un collègue vous tend un tenseur de forme $(3, 64, 384, 384)$ et vous dit que c'est une vidéo. Que représente chaque nombre ? Quelle est la convention utilisée, et quelle serait la forme du même tenseur dans l'autre convention ?

Solution

Le 3 ne peut être que le nombre de canaux de couleur : aucune vidéo ne fait trois pixels de haut. Il s'agit donc de l'ordre (canaux, temps, hauteur, largeur), celui de PyTorch : une vidéo en couleur de 64 images en 384×384. Dans la convention d'affichage, le même tenseur s'écrirait $(64, 384, 384, 3)$. Les nombres qu'il contient sont exactement les mêmes ; seul leur rangement change.

CHAPITRE 2

Ce que veut dire « apprendre » pour une machine

Nous savons maintenant qu'une image n'est qu'un grand tas de nombres. Mais un tas de nombres ne fait rien tout seul. Dans ce chapitre, nous allons voir comment une machine part d'un réglage complètement au hasard, se trompe, remarque qu'elle s'est trompée, et se corrige toute seule. C'est cela, et rien d'autre, qu'on appelle « apprendre ».

> **Ce qu'il faut savoir avant**
>
> Le chapitre 1, et seulement lui. Il faut savoir qu'une image est une grille de nombres, et savoir ce qu'est la forme d'un tenseur. Pour le reste, il suffit de savoir additionner, multiplier, et diviser. Nous n'utiliserons aucune formule que nous n'aurons pas d'abord dessinée.

Quand on dit qu'une machine « apprend », on imagine souvent quelque chose de mystérieux, presque magique. La réalité est beaucoup plus simple, et beaucoup plus rassurante : une machine qui apprend, c'est une machine qui essaie, qui se trompe, qui regarde de combien elle s'est trompée, et qui recommence un tout petit peu mieux. Des milliers de fois. Des millions de fois.

Tout ce chapitre raconte cette boucle-là. Nous allons la suivre pas à pas sur un exemple minuscule, que nous calculerons entièrement à la main, avant de vérifier sur une vraie machine que nos calculs de papier tombent juste.

2.1 Une machine qui apprend, c'est une machine à boutons

En clair

Imaginez un appareil avec quelques boutons à tourner. Vous lui posez une question, il donne une réponse. Si la réponse est mauvaise, vous tournez un peu les boutons et vous redemandez. Apprendre, pour une machine, c'est exactement cela : trouver la bonne position des boutons.

Vous avez sûrement déjà réglé la température d'une douche. Il y a un robinet chaud et un robinet froid. Vous les tournez un peu, vous mettez la main sous l'eau, c'est trop froid, alors vous tournez encore. Au bout de quelques essais, l'eau est parfaite. Personne ne vous a donné la position exacte des robinets : vous l'avez trouvée en essayant et en corrigeant.

Une machine qui apprend fait la même chose, sauf qu'elle a beaucoup plus de robinets, et qu'elle ne trempe pas la main dans l'eau : elle calcule.

Définition 2.1 (Le modèle)

Un *modèle* est une machine à calculer qui prend quelque chose en entrée et rend une réponse en sortie. Ce qu'elle rend ne dépend pas seulement de l'entrée : cela dépend aussi de la position de ses boutons. Le même modèle, avec des boutons réglés autrement, donnera une autre réponse à la même question.

Définition 2.2 (Le paramètre, ou poids)

Un *paramètre* est un de ces boutons. C'est simplement un nombre, rangé dans la mémoire de la machine, que l'on a le droit de changer. On l'appelle aussi un *poids*, parce qu'il décide de l'importance que la machine donne à telle ou telle information. Régler un modèle, c'est choisir la valeur de tous ses paramètres.

La figure 2.1 montre cette idée dans son entier : une entrée qui arrive, des boutons qu'on peut tourner, une réponse qui sort.

Prenons tout de suite un exemple, et gardons-le jusqu'à la fin du chapitre.

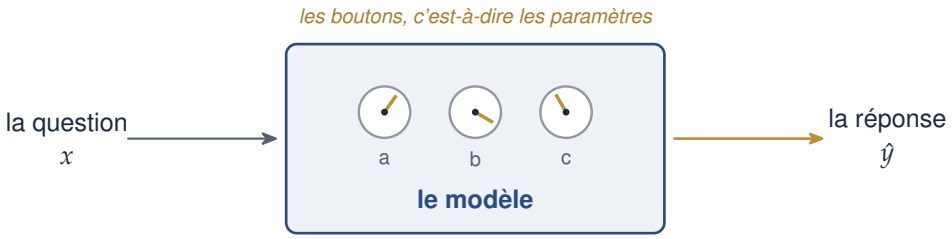


FIGURE 2.1. Un modèle, c'est une boîte à calculer munie de boutons. On lui donne une entrée par la gauche, elle rend une réponse par la droite. Entre les deux, ce sont les boutons — les paramètres — qui décident du résultat. Tourner les boutons change la réponse sans rien changer à la question.

Exemple 2.3 (La machine à bonbons)

Dans le hall d'une école, il y a une machine à bonbons. On y glisse des pièces, et elle rend des bonbons. Trois enfants ont essayé et ont noté ce qu'ils ont obtenu :

Enfant	Pièces glissées	Bonbons reçus
Léa	1	2
Sami	2	5
Nina	3	8

Nous aimerions une machine à calculer qui devine le nombre de bonbons à partir du nombre de pièces. Appelons x le nombre de pièces et $\hat{y}$ le nombre de bonbons deviné. Le modèle le plus simple du monde s'écrit

$$\hat{y} = a \times x + b.$$

Il a exactement deux boutons : a et b. Le bouton a dit combien de bonbons on gagne par pièce ; le bouton b ajoute ou enlève un nombre fixe de bonbons, quel que soit le nombre de pièces.

L'intuition

Le bouton a, c'est la pente : plus il est grand, plus la machine est généreuse à chaque pièce. Le bouton b, c'est le cadeau de départ : il

décale toute la réponse vers le haut ou vers le bas, même quand on n'a mis aucune pièce.

Vous avez peut-être déjà trouvé la règle de tête : trois bonbons par pièce, moins un bonbon que la machine garde toujours pour elle. Autrement dit $a = 3$ et $b = -1$. Vérifions sur les trois enfants : $3 \times 1 - 1 = 2$ pour Léa, $3 \times 2 - 1 = 5$ pour Sami, $3 \times 3 - 1 = 8$ pour Nina. Tout tombe juste.

Piège à éviter

Nous connaissons la bonne réponse, mais **la machine, elle, ne la connaît pas**. C'est tout l'intérêt de l'exercice : nous allons la laisser partir de boutons réglés à zéro, complètement à côté de la plaque, et la regarder retrouver toute seule le 3 et le -1. Connaître la réponse à l'avance nous sert seulement à vérifier qu'elle ne se trompe pas de chemin.

Définition 2.4 (La prédiction)

Une *prédiction* est la réponse que le modèle donne avec les boutons tels qu'ils sont réglés en ce moment. On la note $\hat{y}$, avec un petit chapeau, pour bien la distinguer de la vraie valeur y que l'on a observée dans la réalité.

Exemple 2.5 (Une première prédiction, très mauvaise)

Réglons les deux boutons à zéro : $a = 0$ et $b = 0$. Pour Léa, qui a mis une pièce :

$$\hat{y} = 0 \times 1 + 0 = 0.$$

La machine annonce zéro bonbon, alors que Léa en a reçu deux. Elle se trompe donc de deux bonbons. Pour Sami elle annonce encore zéro, alors qu'il en a reçu cinq. Et pour Nina, zéro contre huit. Notre modèle est pour l'instant parfaitement nul, et c'est normal : nous n'avons rien réglé du tout.

Nous avons une machine, des boutons, et des réponses fausses. Il nous manque quelque chose d'essentiel : un moyen de dire *à quel point* elles sont

fausses.

2.2 Se tromper, et mesurer de combien

En clair

Pour se corriger, il ne suffit pas de savoir qu'on s'est trompé. Il faut un nombre qui dise si l'on s'est trompé beaucoup ou un tout petit peu. Ce nombre s'appelle l'erreur.

Reprenons Léa. La machine a dit 0, la réalité était 2. L'écart entre les deux vaut $0 - 2 = -2$. Nous pourrions nous arrêter là et additionner les écarts des trois enfants. Mais un problème apparaît aussitôt.

Piège à éviter

Additionner les écarts tels quels ne marche pas. Imaginez une machine qui se trompe de $+5$ bonbons pour Léa et de -5 bonbons pour Sami. En additionnant, on obtient $5 + (-5) = 0$: on croirait que la machine est parfaite, alors qu'elle s'est lourdement trompée deux fois ! Les erreurs se sont annulées entre elles.

Il faut donc empêcher les écarts de se compenser. La solution retenue presque partout est de **mettre chaque écart au carré**, c'est-à-dire de le multiplier par lui-même. Un carré est toujours positif : $(+5)^2 = 25$ et $(-5)^2 = 25$ aussi. Plus rien ne s'annule.

L'intuition

Mettre au carré a un deuxième avantage, qui est même le principal : cela punit très fort les grosses erreurs. Se tromper de 1 coûte 1. Se tromper de 2 ne coûte pas 2 mais 4. Se tromper de 10 coûte 100. La machine va donc s'occuper en priorité de ses plus grosses bêtises, ce qui est exactement ce qu'on veut.

Définition 2.6 (L'erreur, ou perte)

L'*erreur* est un seul nombre qui résume à quel point le modèle se trompe sur l'ensemble des exemples. On l'appelle aussi la *perte*, et on la note souvent $\mathcal{L}$, de l'anglais *loss*. On la calcule en mettant chaque écart au carré, puis en prenant la moyenne :

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2,$$

où N est le nombre d'exemples, $\hat{y}_i$ la réponse du modèle sur l'exemple numéro i, et y_i la vraie valeur. Une erreur de 0 veut dire que le modèle tombe juste partout. Plus le nombre est grand, plus le modèle est mauvais.

Notation 2.7 (Lire le signe somme)

Le grand symbole $\sum$ se lit « somme de ». L'écriture $\sum_{i=1}^{3} z_i$ veut simplement dire $z_1 + z_2 + z_3$: on fait prendre à i toutes les valeurs de 1 à 3, et on additionne tout. C'est une façon courte d'écrire une addition longue, rien de plus. Si l'addition ne comporte que trois termes, vous avez parfaitement le droit de les écrire à la main : c'est ce que nous ferons.

Exemple 2.8 (L'erreur de notre machine réglée à zéro)

Avec $a = 0$ et $b = 0$, les trois prédictions valent 0. Calculons les écarts, puis leurs carrés :

Enfant	x	y	$\hat{y}$	écart $\hat{y} - y$	carré
Léa	1	2	0	-2	4
Sami	2	5	0	-5	25
Nina	3	8	0	-8	64

La somme des carrés vaut $4 + 25 + 64 = 93$. Comme il y a trois exemples, la moyenne vaut

$$\mathcal{L} = \frac{93}{3} = 31.$$

Retenez ce nombre : **l'erreur de départ vaut exactement** 31. Nous allons passer le reste du chapitre à la faire descendre.

La figure 2.2 donne à voir ce que ces écarts représentent : ce sont des distances verticales entre les vrais points et la droite que dessine le modèle.

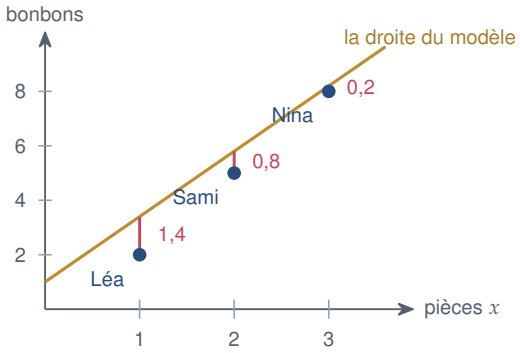


FIGURE 2.2. Les trois enfants sont les points bleus. La droite en trait plein est la réponse du modèle : elle donne un bonbon deviné pour chaque nombre de pièces. Les traits rouges verticaux sont les écarts, c'est-à-dire ce que la machine s'est trompée sur chaque enfant. Régler les boutons, c'est faire pivoter et glisser cette droite jusqu'à ce que les traits rouges soient les plus courts possible.

À retenir

Toute la suite du livre repose sur ce couple d'idées : un modèle donne une réponse, et une erreur mesure de combien cette réponse est fausse. Apprendre, ce sera toujours et uniquement : **faire descendre l'erreur en changeant les paramètres**.

Nous savons mesurer notre bêtise. Reste la question qui compte vraiment : dans quel sens faut-il tourner les boutons pour que ce nombre baisse ?

2.3 De quel côté tourner le bouton ? La pente

En clair

Pour savoir dans quel sens tourner un bouton, il suffit de le pousser un tout petit peu et de regarder ce que devient l'erreur. Si elle monte, on tournait du mauvais côté. Si elle descend, on continue.

C'est le jeu du « tu chauffes, tu refroidis ». On cache un objet dans une pièce, l'enfant fait un pas, et on lui dit s'il se rapproche ou s'il s'éloigne. Avec cette seule information, répétée assez souvent, il finit toujours par trouver.

Faisons-le pour de vrai sur notre machine à bonbons. Nos boutons sont à $a = 0$ et $b = 0$, et l'erreur vaut 31. Poussons le bouton a d'un tout petit peu, disons de 0,01, sans toucher à b.

Exemple 2.9 (Pousser un bouton et regarder)

Avec $a = 0{,}01$ et $b = 0$, les trois prédictions deviennent 0,01, 0,02 et 0,03. Les écarts valent donc $-1{,}99$, $-4{,}98$ et $-7{,}97$. Leurs carrés valent 3,9601, 24,8004 et 63,5209, dont la somme fait 92,2814. La nouvelle erreur vaut

$$\mathcal{L} = \frac{92{,}2814}{3} \approx 30{,}7605.$$

Elle est passée de 31 à 30,7605 : elle a **baissé** de 0,2395. Nous poussions donc dans le bon sens. Pour savoir à quelle vitesse elle baisse, on divise la baisse par la taille de la poussée :

$$\frac{-0{,}2395}{0{,}01} = -23{,}95.$$

Ce nombre, $-23{,}95$, dit deux choses à la fois. Son *signe* négatif dit : « en augmentant a, l'erreur diminue ». Sa *taille*, presque 24, dit : « et elle diminue vite ».

Définition 2.10 (La pente)

La *pente* par rapport à un bouton est le nombre qui dit de combien l'erreur change quand on pousse ce bouton d'une unité. Une pente

négative veut dire que l'erreur descend quand on augmente le bouton. Une pente positive veut dire qu'elle monte. Une pente proche de zéro veut dire que ce bouton ne change presque rien.

Définition 2.11 (Le gradient)

Le *gradient* est la liste de toutes les pentes, une par bouton. S'il y a deux boutons, le gradient contient deux nombres ; s'il y en a deux milliards, il en contient deux milliards. Le gradient est donc une sorte de boussole : il indique, pour chaque bouton à la fois, dans quel sens tourner.

Pousser chaque bouton à la main pour mesurer sa pente serait épouvantable : un modèle comme V-JEPA 2.1 a deux milliards de boutons, il faudrait deux milliards de petites poussées à chaque étape. Heureusement, les mathématiques savent calculer ces pentes directement, d'un coup, par une formule. Pour notre droite, les deux formules sont les suivantes :

Notation 2.12 (Les deux pentes de notre modèle)

En notant $e_i = \hat{y}_i - y_i$ l'écart sur l'exemple i :

$$\text{pente selon } a \;=\; \frac{2}{N}\sum_i e_i \times x_i, \qquad \text{pente selon } b \;=\; \frac{2}{N}\sum_i e_i.$$

Ces deux formules ont l'air de tomber du ciel. Elles ne sont pourtant pas mystérieuses, et il vaut la peine de comprendre ce qu'elles racontent.

L'intuition

Regardez la pente selon b : c'est simplement la moyenne des écarts, multipliée par deux. C'est logique, car le bouton b déplace toute la droite vers le haut ou vers le bas, de la même quantité pour tout le monde. Si en moyenne la machine annonce trop de bonbons, il faut baisser b.

La pente selon a est la même chose, mais chaque écart y est d'abord multiplié par x_i, le nombre de pièces. C'est l'effet du levier. Le bouton a fait pivoter la droite autour du point de départ : un enfant placé loin,

comme Nina avec ses trois pièces, monte ou descend trois fois plus qu'un enfant placé près, comme Léa. Son écart pèse donc trois fois plus lourd dans la décision.

Exemple 2.13 (La formule donne-t-elle le même résultat que la poussée ?)

Vérifions. Avec $a = 0$ et $b = 0$, les écarts valent -2, -5 et -8, et les pièces valent 1, 2 et 3. La pente selon a vaut donc

$$\frac{2}{3}\big[(-2)\times 1+(-5)\times 2+(-8)\times 3\big] = \frac{2}{3}\big[-2-10-24\big] = \frac{2}{3}\times(-36) = -24.$$

La formule annonce -24, et notre petite poussée avait mesuré $-23{,}95$. Les deux méthodes s'accordent. Le minuscule écart vient de ce que notre poussée de 0,01 n'était pas infiniment petite ; en poussant encore moins fort, on collerait encore mieux au -24.
Calculons aussi la pente selon b :

$$\frac{2}{3}\big[(-2)+(-5)+(-8)\big] = \frac{2}{3}\times(-15) = -10.$$

Les deux pentes sont négatives. La boussole nous dit donc : augmente a, et augmente b aussi. Voyons maintenant comment transformer ce conseil en mouvement.

2.4 La descente de gradient, calculée à la main

En clair

La règle tient en une phrase : on tourne chaque bouton dans le sens opposé à sa pente, d'une petite quantité. Puis on recommence.

Pourquoi le sens *opposé* ? Parce qu'une pente négative signifie « en augmentant, l'erreur descend ». Prendre l'opposé du négatif donne du positif : on augmente. Et une pente positive signifie « en augmentant, l'erreur monte » ; son opposé nous fait diminuer. Dans les deux cas, on va vers le bas. C'est la même règle qu'une bille lâchée sur une colline : elle part toujours dans le sens contraire de la montée.

Définition 2.14 (La descente de gradient)

La *descente de gradient* est la méthode qui consiste à répéter l'opération suivante : calculer l'erreur, calculer le gradient, puis déplacer chaque paramètre d'un petit pas dans le sens opposé à sa pente. Pour un paramètre p quelconque, la mise à jour s'écrit

$$p_{\text{nouveau}} = p_{\text{ancien}} - \eta \times \text{pente selon } p,$$

où η est un petit nombre positif choisi par nous, qui règle la longueur du pas.

La figure 2.3 montre cette idée sous sa forme la plus parlante : l'erreur dessine une vallée, et le modèle est une bille qui la descend.

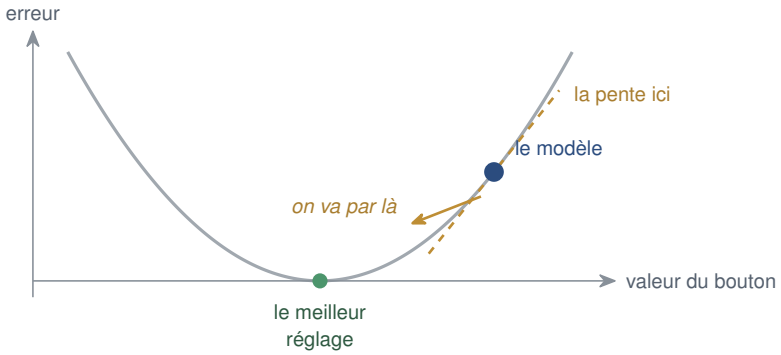


FIGURE 2.3. L'erreur vue comme une vallée. En allant vers la droite ou vers la gauche, on change la valeur d'un bouton ; la hauteur de la courbe donne l'erreur correspondante. La bille est notre modèle. La droite en pointillés est la pente à l'endroit où se trouve la bille : elle monte vers la droite, donc la pente est positive, donc il faut aller à gauche. Le meilleur réglage est tout en bas de la vallée, là où la courbe est plate.

Nous avons tout ce qu'il faut. Choisissons une longueur de pas de 0,1 et faisons deux étapes complètes, en écrivant chaque calcul.

Exemple 2.15 (Première étape, tout en détail)

Nous partons de $a = 0$ et $b = 0$. Nous avons déjà tout calculé : l'erreur vaut 31, la pente selon a vaut -24, la pente selon b vaut -10.

Appliquons la règle avec un pas de 0,1 :

$$a_{\text{nouveau}} = 0 - 0{,}1 \times (-24) = 0 + 2{,}4 = 2{,}4,$$

$$b_{\text{nouveau}} = 0 - 0{,}1 \times (-10) = 0 + 1 = 1.$$

Nos deux boutons sont maintenant réglés à $a = 2{,}4$ et $b = 1$.

Exemple 2.16 (A-t-on vraiment progressé ?)

Vérifions en recalculant l'erreur avec les nouveaux boutons. Les trois prédictions valent :

$$2{,}4 \times 1 + 1 = 3{,}4; \quad 2{,}4 \times 2 + 1 = 5{,}8; \quad 2{,}4 \times 3 + 1 = 8{,}2.$$

Les vraies valeurs étaient 2, 5 et 8. Les écarts valent donc 1,4, 0,8 et 0,2, et leurs carrés 1,96, 0,64 et 0,04. La nouvelle erreur vaut

$$\mathcal{L} = \frac{1{,}96 + 0{,}64 + 0{,}04}{3} = \frac{2{,}64}{3} = 0{,}88.$$

Nous sommes passés de 31 à 0,88 en une seule étape. L'erreur a été divisée par plus de trente-cinq. La machine, qui annonçait zéro bonbon à tout le monde, en annonce maintenant trois et demi à Léa et huit à Nina : elle a déjà presque compris.

Exemple 2.17 (Deuxième étape)

Recommençons exactement le même travail depuis $a = 2{,}4$ et $b = 1$. Les écarts, que nous venons de calculer, valent 1,4, 0,8 et 0,2. Ils sont maintenant positifs : la machine annonce trop de bonbons.
Pente selon a :

$$\frac{2}{3}\big[1{,}4 \times 1 + 0{,}8 \times 2 + 0{,}2 \times 3\big] = \frac{2}{3}\big[1{,}4 + 1{,}6 + 0{,}6\big] = \frac{2}{3} \times 3{,}6 = 2{,}4.$$

Pente selon b :

$$\frac{2}{3}\big[1{,}4 + 0{,}8 + 0{,}2\big] = \frac{2}{3} \times 2{,}4 = 1{,}6.$$

Les deux pentes sont cette fois positives : il faut donc *baisser* les deux boutons.

$$a_{\text{nouveau}} = 2{,}4 - 0{,}1 \times 2{,}4 = 2{,}4 - 0{,}24 = 2{,}16,$$

$$b_{\text{nouveau}} = 1 - 0{,}1 \times 1{,}6 = 1 - 0{,}16 = 0{,}84.$$

Et la nouvelle erreur vaut 0,496, contre 0,88 juste avant : elle a encore baissé.

Récapitulons les deux étapes dans un seul tableau, que vous pourrez comparer tout à l'heure à la sortie du programme.

Étape	a	b	**erreur**	**pente selon** a	**pente selon** b
0	0	0	31	-24	-10
1	2,4	1	0,88	2,4	1,6
2	2,16	0,84	0,496	$-0{,}48$	0,32

Piège à éviter

Regardez la colonne du bouton a : il monte à 2,4, puis *redescend* à 2,16, alors que la bonne réponse est 3. On dirait qu'il s'éloigne ! Ne vous inquiétez pas, et surtout ne jugez jamais un entraînement en surveillant un paramètre isolé. Les boutons bougent ensemble, ils se gênent et se rattrapent l'un l'autre. Le seul chiffre qui compte est l'erreur, et elle, elle descend sans arrêt : 31, puis 0,88, puis 0,496. Au bout de deux cents étapes, a vaut 2,993 et b vaut $-0{,}985$: la machine a bien retrouvé le 3 et le -1 que nous cherchions.

À quoi ça sert en IA ?

Ce que nous venons de faire à la main sur deux boutons est *exactement* ce que fait l'entraînement de V-JEPA 2.1, sans un seul changement d'idée. Les seules différences sont des différences de taille : deux milliards de boutons au lieu de deux, une erreur calculée sur des morceaux de vidéo au lieu de trois enfants, et cent trente-cinq mille étapes au lieu

de deux. La boucle, elle, est identique : je prédis, je mesure mon erreur, je calcule mes pentes, je fais un petit pas, je recommence.

Il nous reste à parler du seul réglage que nous avons choisi arbitrairement, et qui est pourtant le plus délicat de tous : la longueur du pas.

2.5 La taille des pas : le taux d'apprentissage

Définition 2.18 (Le taux d'apprentissage)

Le *taux d'apprentissage*, noté η, est le petit nombre par lequel on multiplie la pente avant de bouger un paramètre. Il décide de la longueur des pas. C'est nous qui le choisissons : la machine ne l'apprend pas.

L'intuition

Vous descendez une colline dans le brouillard. Vous ne voyez que le sol sous vos pieds, donc vous ne connaissez que la pente à l'endroit où vous êtes. Avec des pas minuscules, vous descendrez, mais la nuit tombera avant que vous arriviez. Avec des pas de sept lieues, vous sauterez par-dessus la vallée et vous vous retrouverez plus haut qu'avant, sur l'autre versant. Il existe une bonne taille de pas entre les deux, et la trouver fait partie du métier.

La figure 2.4 met les trois situations côte à côte.

Ce ne sont pas des dessins pour faire joli : refaisons une seule étape de notre machine à bonbons avec trois pas différents, et regardons ce que devient l'erreur.

Exemple 2.19 (Trois pas, trois destins)

Nous partons toujours de $a = 0$, $b = 0$, avec une erreur de 31 et des pentes de -24 et -10. Une seule étape suffit à tout révéler.



FIGURE 2.4. Trois façons de descendre la même vallée. À gauche, les pas sont trop courts : la bille avance à peine et il faudra une éternité. Au milieu, les pas sont bien choisis : la bille rejoint le fond en quelques sauts. À droite, les pas sont trop longs : la bille saute par-dessus le fond, remonte plus haut de l'autre côté, et s'échappe de la vallée au lieu d'y descendre.

Pas η	nouveau a	nouveau b	erreur après une étape
0,001	0,024	0,01	30,33
0,1	2,4	1	**0,88**
1	24	10	3 103

Avec un pas de 0,001, l'erreur passe de 31 à 30,33 : elle a bougé d'à peine deux pour cent. À ce rythme, il faudrait des milliers d'étapes. Avec un pas de 0,1, elle tombe à 0,88 : c'est le bon réglage. Avec un pas de 1, elle grimpe à 3 103, soit cent fois pire qu'au départ ! Et si l'on s'obstine, l'étape suivante donne une erreur de 316 063. La machine ne se contente pas d'échouer : elle s'envole.

Piège à éviter

Un taux d'apprentissage trop grand est la panne numéro un des débutants, et elle est facile à reconnaître : au lieu de descendre doucement, l'erreur grossit d'étape en étape, jusqu'à afficher des nombres absurdes, voire le mot `nan`, qui veut dire « ce n'est plus un nombre du tout ». Le réflexe correct, dans ce cas, est presque toujours le même : diviser le taux d'apprentissage par dix et relancer.

Sur des données réelles

V-JEPA 2.1 ne garde pas le même pas du début à la fin. Pendant les douze mille premières étapes, il l'augmente doucement de 10^{-4} à $5{,}25 \times 10^{-4}$: on appelle cela une montée en douceur, et cela évite de tout casser au tout début, quand les boutons sont encore n'importe comment. Ensuite il le laisse constant très longtemps. Enfin, sur les douze mille dernières étapes, il le fait redescendre jusqu'à 10^{-6}, c'est-à-dire presque zéro : les pas deviennent minuscules et le modèle se pose tout doucement au fond de la vallée, au lieu de continuer à sautiller autour. Cette dernière phase porte un nom que nous retrouverons souvent : la phase de refroidissement.

Nous savons faire un pas, et le faire de la bonne longueur. Il reste à savoir sur combien de données on le calcule, et combien de fois on recommence.

2.6 Le rythme de l'entraînement : le lot et l'époque

Jusqu'ici, nous avions trois enfants, et nous avons calculé l'erreur sur les trois à chaque étape. Avec trois exemples, c'est facile. Avec cent quarante-deux millions d'images, c'est impossible : regarder toute la collection avant de bouger un seul bouton prendrait des jours pour une seule étape.

En clair

La solution est simple et un peu paresseuse : au lieu de regarder toutes les données, on en prend une petite poignée au hasard, on calcule l'erreur sur cette poignée-là, et on fait quand même un pas. La direction est un peu approximative, mais on avance beaucoup plus souvent — et au bout du compte, on va bien plus vite.

Définition 2.20 (Le lot)

Un *lot* est la petite poignée d'exemples que l'on regarde ensemble pour calculer une erreur et faire un pas. En anglais on dit *batch*, et ce mot revient partout dans le code. Le nombre d'exemples dans la poignée s'appelle la *taille de lot*.

Définition 2.21 (Le pas, ou itération)

Un *pas* — on dit aussi une *itération* — est un tour complet de la boucle : on prend un lot, on prédit, on mesure l'erreur, on calcule les pentes, on bouge les paramètres. Un pas, un lot.

Définition 2.22 (L'époque)

Une *époque* est le temps qu'il faut pour que le modèle ait vu une fois chacune des données disponibles. Si la collection contient mille images et que les lots en contiennent cent, alors une époque vaut dix pas.

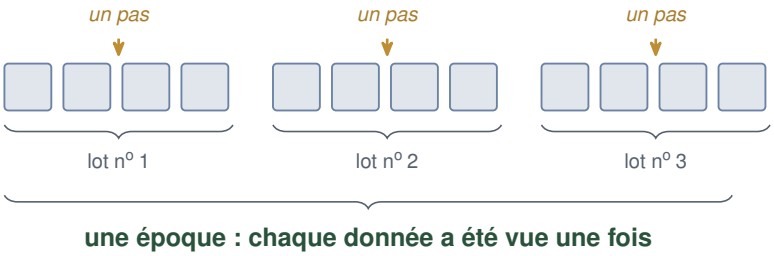


FIGURE 2.5. Toutes les données de la collection, découpées en lots. Le modèle en prend un, fait un pas, en prend un autre, refait un pas. Quand il a épuisé tous les lots, il a vu chaque donnée une fois : cela s'appelle une époque, et il peut alors recommencer un nouveau tour.

Exemple 2.23 (Compter des époques sur un petit jeu)

Vous disposez de 1 000 photos et vous choisissez des lots de 100 photos. Chaque pas consomme 100 photos, il faut donc

$$\frac{1\,000}{100} = 10 \text{ pas pour faire une époque.}$$

Si vous entraînez pendant 50 époques, votre programme aura effectué $50 \times 10 = 500$ pas au total, et chaque photo aura été vue 50 fois.

Sur des données réelles

Faisons le même calcul pour V-JEPA 2.1, et le résultat va sans doute vous surprendre. Le modèle fait 135 000 pas dans sa phase principale, et chaque lot contient 2 304 images. Il a donc regardé

$$135\,000 \times 2\,304 = 311\,040\,000 \text{ images en tout,}$$

soit trois cent onze millions. Or sa collection d'images en compte 142 millions. Le nombre d'époques vaut donc

$$\frac{311\,040\,000}{142\,000\,000} \approx 2{,}2.$$

Chaque image n'est vue qu'un peu plus de deux fois dans toute la vie du modèle. Du côté des vidéos, c'est encore plus frappant : 135 000 pas avec 128 clips par lot font 17 280 000 clips, pour une collection de 19 millions de vidéos. Le modèle ne voit donc même pas une fois chaque vidéo !

C'est pour cela que les articles de recherche modernes ne parlent presque plus d'époques, mais comptent en pas. Quand les données sont quasi inépuisables, repasser deux fois sur la même image n'a plus grand intérêt : mieux vaut en voir une nouvelle.

Pour aller plus loin

Prendre un lot au hasard plutôt que toutes les données rend le gradient un peu bruité : la direction indiquée n'est pas tout à fait la bonne, elle tremble. On appelle cela la descente de gradient *stochastique*, un mot savant qui veut simplement dire « au hasard ». Ce tremblement, contre toute attente, aide souvent : il empêche la bille de rester coincée dans un petit creux sans intérêt et lui fait trouver de meilleures vallées.

Nous savons entraîner. Il reste un dernier danger, et c'est le plus sournois de tous : une machine peut très bien faire baisser son erreur tout en n'ayant strictement rien compris.

2.7 Apprendre par cœur n'est pas apprendre

En clair

Un élève peut apprendre par cœur les réponses des exercices du livre sans comprendre la leçon. Tant qu'on l'interroge sur ces exercices-là, il a tout juste. Le jour où on lui pose une question nouvelle, il s'effondre. Une machine peut faire exactement pareil.

Revenons à nos trois enfants. Notre droite n'a que deux boutons : elle est trop simple pour tricher, elle est bien obligée de trouver une règle générale. Mais donnons à un modèle beaucoup, beaucoup de boutons, et il pourra faire passer une courbe tordue exactement par chacun des trois points. Son erreur tombera à zéro. Et pourtant, si l'on glisse quatre pièces dans la machine, ce modèle-là annoncera n'importe quoi.

Définition 2.24 (Le sur-apprentissage)

Le *sur-apprentissage* est ce qui arrive quand un modèle apprend par cœur les exemples qu'on lui a montrés, y compris leurs petits défauts, au lieu d'apprendre la règle générale qui se cache derrière. Il est alors excellent sur ce qu'il a déjà vu, et mauvais sur tout le reste.

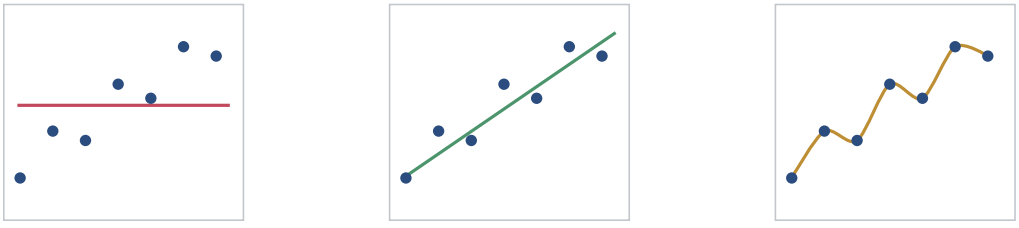


FIGURE 2.6. Les mêmes points, expliqués de trois façons. À gauche, le modèle est trop simple : il ne suit même pas la tendance, il passe à côté de tout le monde. Au milieu, il a saisi la règle générale : il ne touche pas tous les points, et c'est très bien ainsi. À droite, il passe exactement par chaque point, son erreur est nulle — mais entre les points il fait n'importe quoi, et sur un point nouveau il se trompera lourdement.

Comment se protéger de ce piège ? En trichant le moins possible : on cache au modèle une partie des données, et on ne les lui montre jamais pendant l'entraînement. Elles serviront uniquement à le juger.

Définition 2.25 (Jeu d'entraînement et jeu de validation)

Le *jeu d'entraînement* rassemble les données que le modèle a le droit de voir pour régler ses boutons. Le *jeu de validation*, parfois appelé jeu de test, rassemble des données qu'il ne voit jamais pendant l'entraînement, et qui servent uniquement à mesurer honnêtement ce qu'il vaut. C'est la différence entre les exercices du cahier et l'examen de fin d'année.

Piège à éviter

La faute la plus grave, dans tout l'apprentissage automatique, est de régler quoi que ce soit en regardant le jeu de validation. Si vous essayez trente taux d'apprentissage et que vous gardez celui qui donne le meilleur score de validation, alors ce score n'est plus honnête : vous avez, sans le vouloir, appris par cœur votre examen. Les vrais résultats se mesurent toujours sur des données que rien ni personne n'a servi à choisir.

À quoi ça sert en IA ?

L'article de V-JEPA 2.1 prend cette précaution très au sérieux, et cela se voit dans ses annexes. Pour le suivi d'objets dans les vidéos, par exemple, les auteurs cherchent les meilleurs réglages sur la partie entraînement du jeu DAVIS, puis appliquent tels quels ces réglages à tous les jeux d'évaluation, sans plus jamais y toucher. C'est un détail d'annexe, et pourtant c'est ce détail qui rend leurs chiffres croyables.

Nous avons maintenant tout le vocabulaire. Le moment est venu de vérifier que nos calculs de papier tiennent devant une vraie machine.

2.8 Passons au code

Nous allons refaire, en Python, les deux étapes que nous venons de calculer à la main. Nous utiliserons `torch`, la bibliothèque PyTorch, qui sait faire deux

choses très utiles : manipuler des tenseurs, et surtout calculer les pentes toute seule. Nous n'aurons donc plus jamais à écrire les formules du gradient nous-mêmes.

Commençons par le plus important : vérifier que la petite poussée du début du chapitre donne bien ce que nous avions trouvé.

PYTHON • Mesurer une pente en poussant un bouton

```python
import torch

# Les trois enfants : pieces glissees et bonbons recus.
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])

def erreur(a, b):
    prediction = a * x + b
    return ((prediction - y) ** 2).mean().item()

avant = erreur(0.00, 0.0)   # les boutons a zero
apres = erreur(0.01, 0.0)   # on pousse le bouton a de 0,01

print(f"erreur avant la poussee : {avant:.4f}")   # -> 31.0000
print(f"erreur apres la poussee : {apres:.4f}")   # -> 30.7605
print(f"pente mesuree           : {(apres - avant) / 0.01:.2f}")
↪  # -> -23.95
```

Les trois nombres affichés sont exactement ceux de notre exemple : 31, 30,7605 et −23,95. Le calcul à la main et la machine sont d'accord.

Voici maintenant la boucle d'entraînement complète. Elle tient en une quinzaine de lignes, et c'est littéralement la même boucle que celle qui entraîne V-JEPA 2.1 : seules les tailles changent.

PYTHON • Deux étapes de descente de gradient

```python
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])

# Nos deux boutons. « requires_grad » dit a PyTorch :
# surveille ce nombre, je te demanderai sa pente.
```

```
a = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
pas = 0.1

for etape in range(2):
    prediction = a * x + b                          # 1. je predis
    erreur = ((prediction - y) ** 2).mean()         # 2. je mesure mon erreur
    erreur.backward()                               # 3. je demande les pentes

    print(f"etape {etape} : a = {a.item():.2f}, b = {b.item():.2f}, "
          f"erreur = {erreur.item():.4f}, "
          f"pente_a = {a.grad.item():.2f}, pente_b = {b.grad.item():.2f}")

    with torch.no_grad():                           # 4. je bouge les boutons
        a -= pas * a.grad
        b -= pas * b.grad

    a.grad.zero_()                                  # 5. j'efface les pentes
    b.grad.zero_()

print(f"apres 2 etapes : a = {a.item():.2f}, b = {b.item():.2f}")
```

Ce programme affiche exactement ceci :

TEXT • Ce que le programme écrit à l'écran

```
etape 0 : a = 0.00, b = 0.00, erreur = 31.0000, pente_a = -24.00, pente_b = -10.00
etape 1 : a = 2.40, b = 1.00, erreur = 0.8800, pente_a = 2.40, pente_b = 1.60
apres 2 etapes : a = 2.16, b = 0.84
```

Comparez ligne à ligne avec le tableau de la section précédente : le 31, le -24, le -10, le 2,4, le 0,88, le 2,16 et le 0,84 y sont tous. Rien n'a été caché, et la machine n'a rien fait de plus que ce que nous avons fait au crayon.

Piège à éviter

La ligne `a.grad.zero_()` paraît anodine, et pourtant l'oublier est l'une des erreurs les plus courantes chez les débutants. PyTorch *additionne* les pentes qu'il calcule au lieu de les remplacer. Si vous n'effacez pas, l'étape suivante utilisera la somme de deux pentes, puis de trois, et vos pas grandiront tout seuls jusqu'à faire exploser l'entraînement. Le symptôme est le même que celui d'un taux d'apprentissage trop grand, ce qui rend la panne difficile à diagnostiquer.

Laissons enfin tourner la même boucle bien plus longtemps, pour voir la machine retrouver la règle des bonbons.

PYTHON • Deux cents étapes, et la règle apparaît

```python
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])
a = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

for _ in range(200):
    erreur = ((a * x + b - y) ** 2).mean()
    erreur.backward()
    with torch.no_grad():
        a -= 0.1 * a.grad
        b -= 0.1 * b.grad
    a.grad.zero_()
    b.grad.zero_()

print(f"a = {a.item():.3f}   b = {b.item():.3f}")   # -> a = 2.993   b = -0.985
print(f"erreur = {erreur.item():.6f}")              # -> erreur = 0.000032
```

Le bouton a vaut 2,993 et le bouton b vaut $-0{,}985$. La machine a retrouvé, toute seule, la règle que nous avions devinée de tête au début du chapitre : trois bonbons par pièce, moins un pour la machine. Personne ne lui a jamais soufflé la réponse. Elle a seulement descendu la vallée, deux cents fois de suite.

À retenir

Retenez cette boucle en cinq gestes, car c'est la seule qu'il y ait dans tout l'apprentissage automatique, du plus petit exemple au plus gros modèle du monde :

1. je prends un lot de données ;
2. je calcule ma prédiction ;
3. je mesure mon erreur ;
4. je calcule les pentes ;
5. je bouge chaque bouton d'un petit pas dans le sens opposé à sa pente.

Puis je recommence. Cent trente-cinq mille fois, dans le cas de V-JEPA 2.1.

Il manque pourtant quelque chose d'énorme à tout ce que nous venons de faire, et vous l'avez peut-être déjà remarqué. À chaque étape, nous avions besoin de connaître la bonne réponse : les deux bonbons de Léa, les cinq de Sami, les huit de Nina. Sans elles, impossible de calculer le moindre écart, donc impossible de calculer la moindre erreur, donc impossible d'apprendre quoi que ce soit.

Mais qui va écrire la bonne réponse à côté d'un milliard de vidéos ? Personne. Aucune équipe au monde, aucun budget, aucune quantité de patience ne suffirait. C'est précisément le mur contre lequel s'arrête tout ce que nous venons d'apprendre — et c'est le mur que le chapitre suivant va traverser.

Exercices

Exercice 1 — Calculer une erreur

Un modèle prédit $\hat{y} = 4$, $\hat{y} = 7$ et $\hat{y} = 9$, alors que les vraies valeurs sont 3, 9 et 9. Quelle est l'erreur, au sens de la moyenne des carrés des écarts ?

Solution

Les écarts valent $4-3=1$, puis $7-9=-2$, puis $9-9=0$. Leurs carrés valent 1, 4 et 0, dont la somme fait 5. Comme il y a trois exemples,

$$\mathcal{L} = \frac{5}{3} \approx 1{,}67.$$

Remarquez que le deuxième écart, bien que négatif, contribue positivement à l'erreur : c'est tout l'intérêt du carré.

Exercice 2 — Faire un pas

Un paramètre vaut actuellement $p = 5$. On calcule sa pente et l'on trouve $+3$. Le taux d'apprentissage vaut 0,2. Que vaut le paramètre après la mise à jour ? A-t-il augmenté ou diminué, et pourquoi était-ce le bon sens ?

Solution

On applique la règle $p_{\text{nouveau}} = p_{\text{ancien}} - \eta \times \text{pente}$, soit

$$p_{\text{nouveau}} = 5 - 0{,}2 \times 3 = 5 - 0{,}6 = 4{,}4.$$

Le paramètre a diminué. C'était le bon sens, car une pente positive signifie « si j'augmente ce bouton, l'erreur monte ». Pour faire descendre l'erreur, il fallait donc baisser le bouton.

Exercice 3 — Compter des époques

Un jeu de données contient 50 000 images et l'on choisit des lots de 250 images. Combien de pas faut-il pour une époque ? Et si l'on entraîne pendant 30 000 pas, combien d'époques cela représente-t-il ?

Solution

Une époque demande $50\,000/250 = 200$ pas. Avec 30 000 pas au total, on effectue

$$\frac{30\,000}{200} = 150 \text{ époques},$$

et chaque image aura donc été vue 150 fois. C'est un régime très différent de celui de V-JEPA 2.1, qui ne dépasse pas deux époques et demie : quand les données sont peu nombreuses, on repasse dessus souvent, et le risque de sur-apprentissage augmente d'autant.

Exercice 4 — Diagnostiquer une panne

Vous lancez un entraînement et vous voyez défiler ces erreurs : 18,4 puis 91,7 puis 2 340 puis `nan`. Que se passe-t-il très probablement, et quel est le premier réglage à corriger ?

Solution

L'erreur ne descend pas, elle explose : le modèle saute par-dessus le fond de la vallée et remonte de plus en plus haut à chaque étape, jusqu'à sortir des nombres représentables par la machine, ce qui produit le `nan`. C'est la signature d'un taux d'apprentissage trop grand. Le premier geste est de le diviser par dix et de relancer ; si le problème persiste, on divise encore par dix. Attention toutefois : oublier d'effacer les pentes avec `zero_()` produit exactement le même symptôme, et vaut la peine d'être vérifié dans la foulée.

Exercice 5 — Le piège de l'examen

Vous disposez d'un jeu d'entraînement et d'un jeu de validation. Vous essayez cinquante taux d'apprentissage différents et vous annoncez, comme résultat final de votre travail, le meilleur score obtenu sur le jeu de validation. Pourquoi ce chiffre est-il trompeur, et que fallait-il faire ?

Solution

En choisissant le réglage qui donne le meilleur score de validation, vous avez utilisé le jeu de validation pour prendre une décision : il a donc servi à l'entraînement, au sens large. Parmi cinquante essais, le meilleur score est en partie dû à la chance, et il sera plus flatteur que la réalité. La bonne pratique consiste à garder un troisième jeu de données, mis

> de côté dès le départ, que l'on ne regarde qu'une seule et unique fois, tout à la fin, pour annoncer le résultat.

Deuxième partie

Comprendre JEPA et V-JEPA 2, l’ancêtre direct

Troisième partie

Le problème que V-JEPA 2.1 vient résoudre

Quatrième partie

V-JEPA 2.1, ingrédient par ingrédient

Cinquième partie

Les résultats, lus avec honnêteté

Sixième partie

Construire le modèle et le programme d'entraînement

Septième partie

De l'entraînement à la production

Bibliographie

[1] M. ASSRAN et al. « V-JEPA 2 : Self-Supervised Video Models Enable Understanding, Prediction and Planning ». In : *arXiv preprint arXiv :2506.09985* (2025). URL : https://arxiv.org/abs/2506.09985.

[2] L. MUR-LABADIA et al. « V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning ». In : *arXiv preprint* (2026). FAIR at Meta et Universidad de Zaragoza. URL : https://github.com/facebookresearch/vjepa2.